

CSE 220: Signals and Systems Sessional

Practice Set - Fourier Series & Continuous Fourier Transform

Self-generated mock questions, harder than the released sample. For practice only.

This practice set mirrors the format of the official sample (2D-CFT edge detector reference and the epicycle Fourier-series pruning task) but each problem below adds one or two extra sub-tasks, a stricter analysis/plotting requirement, or a variant transform, so treat these as a harder ceiling for what the real exam might ask. Total marks and time per set are modeled after the sample (10 marks / 25 minutes).

Practice Set A — Band-Pass Edge Isolation (2D CFT)

Total Marks: 12 | Time: 30 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

The given `cft_edge_detector.py` template only implements an ideal circular *high-pass* filter (`FrequencyFilter.high_pass`). In this set you must extend the filtering stage so a single narrow annulus (ring) of spatial frequencies can be isolated, and quantify how much image energy lives inside that ring.

Task 1: Implement an Ideal Band-Pass Filter

Add a method `band_pass(self, real, imag, low_cutoff, high_cutoff)` to the `FrequencyFilter` class. Using the same $(i - cx), (j - cy)$ radial-distance convention as `high_pass`, zero out every frequency-domain sample whose radius r satisfies $r < \text{low_cutoff}$ OR $r > \text{high_cutoff}$, leaving only the annulus $\text{low_cutoff} \leq r \leq \text{high_cutoff}$ untouched. Do not use any FFT/DFT routine — operate directly on the `real/imag` arrays already produced by `CFT2D.compute_cft()`.

Task 2: Spectral Energy Ratio

Add a method `spectral_energy_ratio(self, real_full, imag_full, real_filtered, imag_filtered)` that returns the fraction of total spectral energy (sum of magnitude-squared over the whole frequency grid) retained after band-pass filtering, i.e. $\text{sum}(|F_{\text{filtered}}|^2) / \text{sum}(|F_{\text{full}}|^2)$.

Task 3: Multi-Ring Reconstruction & Comparison

In a new script section (or `__main__` block), reconstruct the spatial image (via `InverseCFT2D`) for THREE different annuli: an inner ring (low-frequency band), a middle ring, and an outer ring (high-frequency band). For each ring, take the absolute value of the reconstruction, normalize it to $[0, 1]$, and produce a single figure with three subplots (use `plt.subplots(1, 3, ...)`) titled with the ring's (`low_cutoff`, `high_cutoff`) values and its spectral energy ratio from Task 2.

Execution & Deliverables

- Reuse `ContinuousImage` and `CFT2D` exactly as given; only extend `FrequencyFilter` and add reconstruction/plotting code.
- Load any grayscale image of your choice (or reuse a sample image if one is provided in the exam materials).
- Choose three non-overlapping (`low_cutoff`, `high_cutoff`) pairs that together span roughly 0 to the maximum radius present in the spectrum.
- Print a small table to console: Ring | (low, high) | Energy Ratio, one row per ring.

- Save the 3-subplot comparison figure as band_pass_comparison.png.

Ring	(low, high)	Energy Ratio

inner	(0.0, 8.0)	0.87xx
middle	(8.0, 20.0)	0.09xx
outer	(20.0, 40.0)	0.02xx

Practice Set B — Denoising via Low-Pass CFT + PSNR

Total Marks: 12 | Time: 30 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

Rather than detecting edges, this set uses the same separable 2D CFT machinery to *denoise* an image corrupted with synthetic Gaussian noise, and asks you to quantify reconstruction quality with a standard image-quality metric instead of just visual inspection.

Task 1: Synthetic Noise Injection

Before computing the CFT, add zero-mean Gaussian noise (`np.random.normal`, choose a sigma that is clearly visible, e.g. $\sigma = 0.05$ on a $[0,1]$ -normalized image) to `img.image` in a copy of the `ContinuousImage` object, keeping the clean original for later comparison.

Task 2: Ideal Low-Pass Filter

Add a method `low_pass(self, real, imag, cutoff)` to `FrequencyFilter` that is the complement of the given `high_pass`: it zeros out every frequency sample with radius $r > \text{cutoff}$ and keeps $r \leq \text{cutoff}$ untouched.

Task 3: PSNR Evaluation Sweep

Add a function `compute_psnr(clean, denoised)` using $\text{PSNR} = 10 * \log_{10}(1.0 / \text{MSE})$ (image range is $[0,1]$). For at least 4 different cutoff values, low-pass filter the noisy spectrum, reconstruct the image, clip it to $[0, 1]$, and compute the PSNR against the clean original. Plot PSNR versus cutoff as a line plot (`plt.plot`) with markers, and mark the cutoff that maximizes PSNR.

Execution & Deliverables

- Do not modify `ContinuousImage` or `CFT2D`; only add to `FrequencyFilter` and write new driver/plotting code.
- Use `np.trapezoid-based InverseCFT2D.reconstruct` exactly as given for the reconstruction step.
- Print a table: Cutoff | PSNR (dB), one row per cutoff tested, plus a final line identifying the best cutoff.
- Save the PSNR-vs-cutoff plot as `psnr_sweep.png` and save the best-cutoff denoised image as `denoised_best.png`.

Practice Set C — Energy-Preserving Harmonic Pruning + Amplitude Spectrum (Fourier Series)

Total Marks: 12 | Time: 30 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

This is a strictly harder version of the released sample task. In addition to energy-based pruning and MSE evaluation, you must analyze the amplitude spectrum directly and produce a compression-ratio vs. fidelity curve rather than just four fixed print rows.

Task 1: `prune_harmonics_by_energy(self, r)`

Exactly as the released sample: given target ratio r in $(0,1]$, zero out the coefficients of the least energetic harmonics until the remaining (retained) harmonics account for at least a fraction r of $E_{\text{total}} = \sum_n |c_n|^2$. Return $(\text{num_retained}, \text{actual_ratio})$.

Task 2: `evaluate_reconstruction_error(self)`

Exactly as the released sample: return $\text{MSE} = (1/M) * \sum_i |f(t_i) - \hat{f}(t_i)|^2$ comparing `self.signal` against `self.approximate(self.t)` using the CURRENT (possibly pruned) `self.coefs`.

Task 3: Amplitude Spectrum Plot (new)

Add a method `plot_amplitude_spectrum(self, ax=None)` that plots $|c_n|$ against harmonic index n (a stem or scatter plot) for $n = -N..N$ on a log-scaled y-axis, so the exponential-like decay of amplitude with $|n|$ is visible.

Task 4: Compression Ratio vs. MSE Curve (new)

For at least 8 target ratios spanning $r = 0.80$ up to $r = 1.00$, run the pruning + MSE evaluation, then produce a single figure with two subplots (`plt.subplots(1, 2)`): left subplot = number of harmonics retained vs r ; right subplot = MSE (log-scaled y-axis) vs r . Both should be line plots with markers so the trade-off curve is visible.

Execution & Deliverables

- Load `svgs/heart.svg` with $N = 150$ exactly like the sample, using `load_svg_path` and `num_points=1000`.
- Print the same 4-row table format as the sample ($r=0.96, 0.98, 0.99, 1.00$) PLUS the amplitude-spectrum and compression-ratio analysis above.
- Save the amplitude spectrum as `heart_amplitude_spectrum.png` and the two-subplot trade-off figure as `heart_compression_tradeoff.png`, in addition to the four `heart_pruned_{r}.png` comparison images.

Practice Set D — Phase-Zeroed Reconstruction & Symmetric-Harmonic Filtering (Fourier Series)

Total Marks: 12 | Time: 30 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

This set probes whether you understand what each Fourier coefficient c_n actually encodes (magnitude vs. phase) rather than just pruning by energy. You will build two DIFFERENT coefficient-manipulation methods and compare their effect on the reconstructed drawing.

Task 1: `zero_phase_reconstruction(self)`

Add a method that returns a NEW dict of coefficients where every c_n has its phase set to zero but its magnitude preserved (i.e. replace c_n with $|c_n|$ for every n), without permanently modifying `self.coefs`. Then reconstruct the resulting shape over `self.t` using these magnitude-only coefficients (you may temporarily swap `self.coefs`, reconstruct, then restore the originals, or refactor `approximate()` to accept an optional `coefs` argument).

Task 2: `keep_symmetric_pairs_only(self, k)`

Add a method that keeps ONLY the k harmonic pairs ($n, -n$) with the largest combined energy $|c_n|^2 + |c_{-n}|^2$ (plus $n = 0$ always kept), zeroing every other coefficient in a working copy of `self.coefs`, and returns that filtered dict. This differs from Task 1's pruning because it forces symmetric retention rather than independently ranking every harmonic.

Task 3: Three-Way Visual Comparison

Produce a single figure with 3 subplots (`plt.subplots(1, 3)`) side by side, each plotting the reconstructed (x, y) curve: (a) the full $N=150$ reconstruction (ground truth), (b) the zero-phase reconstruction from Task 1, (c) the symmetric-pair reconstruction from Task 2 with $k = 20$. Report the MSE of (b) and (c) against the true signal in each subplot's title.

Execution & Deliverables

- Load `svgs/heart.svg` with $N = 150$ exactly like the sample.
- Do not permanently overwrite `self.coefs` when computing the zero-phase or symmetric-pair variants unless you restore the originals afterward — downstream code (and the grader) expects `self.coefs` to still hold the full $N=150$ solution when the script finishes.
- Save the 3-subplot comparison as `heart_phase_symmetry_comparison.png` and print the two MSE values to console.

Practice Set E — Notch Filtering for Periodic Noise Removal (2D CFT)

Total Marks: 10 | Time: 25 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

Real scanned images often carry periodic interference (e.g. scan-line or halftone patterns), which shows up as isolated bright spots away from the DC component in the spectrum. Unlike high/low/band-pass, a notch filter removes only small localized regions rather than an entire radial band.

Task 1: Implement a Notch Filter

Add a method **notch(self, real, imag, centers, radius)** to `FrequencyFilter`, where `centers` is a list of (i, j) pixel coordinates in the frequency grid. Zero out every sample within radius of ANY center (using the same $(i-ci)^2 + (j-cj)^2 \leq \text{radius}^2$ test as `high_pass`), leaving everything else untouched.

Task 2: Synthetic Periodic Noise + Recovery

Add sinusoidal stripe noise to a clean image (e.g. `image += 0.15*np.sin(2*np.pi*k*yy)` for some frequency k), compute its CFT, locate the two symmetric bright spots the stripes create away from the DC peak (you may do this by inspection of `plot_magnitude()` or by `np.argmax` on $|F|$ outside a small DC-exclusion radius), and notch them out. Reconstruct and compare the notched result to the clean original with a 1x3 subplot (clean, noisy, notch-filtered).

Execution & Deliverables

- Reuse `ContinuousImage/CFT2D/InverseCFT2D` unmodified; only extend `FrequencyFilter`.
- Print the pixel coordinates you identified as the noise spikes and the radius used.
- Save the 3-panel comparison as `notch_filter_comparison.png`.

Practice Set F — Ideal vs. Gaussian (Soft) Low-Pass Filtering (2D CFT)

Total Marks: 10 | Time: 25 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

Ideal filters (hard cutoffs) cause ringing artifacts in the reconstruction. This set asks you to implement a smooth Gaussian low-pass filter and quantitatively compare it against the ideal low-pass from the given `high_pass`-style masking.

Task 1: Gaussian Low-Pass Filter

Add a method **`gaussian_low_pass(self, real, imag, sigma)`** that multiplies (not zeros/keeps) every frequency sample by a Gaussian weight $\exp(-r^2 / (2 \cdot \sigma^2))$, where r is the same radial distance used in `high_pass`. This should scale `real` and `imag` continuously rather than making a binary keep/discard decision.

Task 2: Ringing Comparison

For a matched cutoff/sigma pair (choose sigma so the Gaussian's half-power point roughly matches an ideal cutoff you pick), reconstruct the image with BOTH filters and plot a 1D horizontal intensity profile (a single row) through both reconstructions on the same axes (`plt.plot`) to visually show the ideal filter's overshoot/ringing near edges versus the Gaussian's smoother roll-off.

Task 3: Quantify Ringing

Add a function **`ringing_score(profile)`** that estimates ringing severity as the sum of magnitudes of local extrema in the 1D profile that occur away from the main edge transition (e.g. using `np.diff` sign changes). Report the score for both filters.

Execution & Deliverables

- Keep the Gaussian filter as elementwise multiplication, not a hard mask, to distinguish it from Sets A/B.
- Save the intensity-profile comparison as `ringing_profile_comparison.png` and print both ringing scores.

Practice Set G — Rotation Behaviour of the 2D CFT Magnitude Spectrum

Total Marks: 10 | Time: 25 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

A core property of the 2D Fourier transform is that rotating the spatial image rotates its magnitude spectrum by the same angle. This set asks you to verify that property numerically using the given separable-integration CFT2D, rather than deriving it analytically.

Task 1: Rotate-and-Compare Pipeline

Using `scipy.ndimage.rotate` (or an equivalent) rotate the loaded image by 30° with `reshape=False`, wrap it back into a `ContinuousImage`-like object with the same x, y axes as the original, and compute its 2D CFT with the given CFT2D class.

Task 2: Quantitative Rotation Check

Add a function `compare_rotated_spectra(mag_original, mag_rotated_image, angle_deg)` that rotates `mag_original` by `angle_deg` using the same rotation utility and returns the normalized cross-correlation (or simply $1 - \text{normalized MSE}$) between that rotated-magnitude prediction and the actually-computed `mag_rotated_image`, as a single float in roughly $[0, 1]$ indicating how well the rotation property held up numerically.

Task 3: Visualization

Produce a 1×3 subplot of: original magnitude spectrum (log-scaled), magnitude spectrum of the rotated image (log-scaled), and the original magnitude spectrum digitally rotated by 30° for visual comparison. Put the similarity score from Task 2 in the figure's subtitle.

Execution & Deliverables

- Use `np.log(1+magnitude)` exactly as in the given `plot_magnitude()` for all magnitude displays.
- Save the figure as `rotation_property_check.png` and print the similarity score.

Practice Set H — Directional (Wedge) Frequency Filtering

Total Marks: 10 | Time: 25 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

Edges in a specific orientation (e.g. mostly-horizontal or mostly-vertical texture) concentrate energy in a wedge-shaped angular sector of the spectrum rather than a full radial ring. This set builds an angular filter to isolate edges of a chosen orientation.

Task 1: Implement a Wedge (Angular) Filter

Add a method `directional_pass(self, real, imag, angle_center_deg, angle_width_deg, min_radius)` to `FrequencyFilter`. For each frequency sample (i, j) , compute its angle $\theta = \text{degrees}(\text{atan2}(i - cx, j - cy))$ and its radius r as in `high_pass`. Keep the sample only if $r \geq \text{min_radius}$ AND its angle lies within $\text{angle_width_deg}/2$ of angle_center_deg OR of $\text{angle_center_deg} + 180$ (the opposite wedge, since real images have Hermitian-symmetric spectra); zero out everything else.

Task 2: Orientation Sweep

For four orientations (0° , 45° , 90° , 135°) with a fixed angular width (e.g. 20°) and a fixed `min_radius`, reconstruct the filtered image for each orientation and display all four in a 2x2 subplot grid (`plt.subplots(2, 2)`), titled by orientation, so the direction-selective edge response is visible.

Execution & Deliverables

- Reuse `InverseCFT2D` exactly as given for every reconstruction; only add to `FrequencyFilter` and driver code.
- Save the 2x2 comparison as `directional_filter_grid.png`.

Practice Set I — Top-K Magnitude Coefficient Compression (2D CFT)

Total Marks: 10 | Time: 25 Minutes | Files: `cft_edge_detector.py` (template)

Context & Objective

Instead of a geometric mask (radius/angle), this set compresses the spectrum by keeping only the K frequency samples with the largest magnitude anywhere on the grid — a simple analogue of transform-coding compression — and studies the reconstruction-quality vs. compression trade-off.

Task 1: Top-K Magnitude Mask

Add a function `top_k_mask(real, imag, k)` that computes $\text{magnitude} = \sqrt{\text{real}^2 + \text{imag}^2}$, finds the k largest-magnitude entries anywhere in the 2D grid (e.g. via `np.argsort` on the flattened array), builds a boolean mask that is True only at those k positions, and returns `(real*mask, imag*mask)`.

Task 2: Compression Ratio vs. PSNR Curve

For at least 6 values of k spanning roughly 1% to 50% of the total number of frequency samples, apply `top_k_mask`, reconstruct with `InverseCFT2D`, clip to `[0,1]`, compute PSNR against the original clean image (reuse or reimplement the PSNR formula from Set B), and plot PSNR (dB) versus the fraction of coefficients kept (`k / total`) as a line plot with markers, x-axis log-scaled.

Execution & Deliverables

- Do not use any built-in image/video compression codec — the masking + reconstruction must go through the same `CFT2D/InverseCFT2D` pipeline as the rest of the assignment.
- Print a small table of k, fraction kept, and PSNR for each tested value, and save the trade-off plot as `topk_compression_tradeoff.png`.

Practice Set J — Gibbs Phenomenon on a Synthesized Square Wave (Fourier Series)

Total Marks: 10 | Time: 25 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

This set moves away from the heart SVG and uses a synthetic real-valued periodic signal (a square wave) to study a classic Fourier-series artifact: the Gibbs overshoot near a jump discontinuity, and how it behaves as more harmonics are added.

Task 1: Build a Square-Wave Signal

Sample a square wave of period $T=2\pi$ over $M=1000$ points as a real-valued (cast to complex) signal: $\text{signal}[i] = +1$ for $t[i]$ in $[0, \pi)$ and -1 for $t[i]$ in $[\pi, 2\pi)$, with t spanning the closed interval $[0, 2\pi]$. Feed this (t, signal) pair into `FourierEpicycles` exactly as `load_svg_path`'s output would be used.

Task 2: Overshoot Measurement vs. N

For N in $\{5, 15, 45, 135\}$, compute all coefficients, reconstruct $\hat{f}(t)$ on a fine time grid, and measure the Gibbs overshoot as $\max(\hat{f}) - 1.0$ (the amount the reconstruction peaks above the true high value near the jump at $t=0/2\pi$). Plot overshoot vs. N (`plt.plot, marker='o'`) and separately plot the four reconstructions overlaid on the true square wave in a single figure to visualize that the overshoot's height stabilizes (~9%) even as N grows.

Execution & Deliverables

- Do not modify `calculate_cn/approximate` signatures; only add a small driver script section.
- Save the overshoot-vs- N plot as `gibbs_overshoot_vs_N.png` and the overlaid reconstructions as `gibbs_reconstructions_overlay.png`.

Practice Set K — Magnitude-Threshold Pruning vs. Energy-Ratio Pruning (Fourier Series)

Total Marks: 10 | Time: 25 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

The released sample prunes by cumulative ENERGY ratio. This set implements a second, simpler pruning rule — an absolute magnitude threshold — and asks you to compare the two strategies directly at matched harmonic counts.

Task 1: `prune_harmonics_by_threshold(self, epsilon)`

Add a method that zeros out every coefficient `c_n` in `self.coeefs` with $|c_n| < \epsilon$ (keep $n=0$ regardless of magnitude), and returns the number of harmonics retained.

Task 2: Matched-Count Comparison

For a fixed harmonics-retained COUNT (e.g. 41), find (a) the smallest r for `prune_harmonics_by_energy` that retains approximately that many harmonics, and (b) the ϵ for `prune_harmonics_by_threshold` that retains approximately the same count (a small search/bisection over ϵ is acceptable). Compute `evaluate_reconstruction_error()` for both and report which strategy achieves lower MSE at the same harmonic budget, with a 1x2 subplot comparing their reconstructed curves against the ground truth.

Execution & Deliverables

- Load `svgs/heart.svg` with $N = 150$ as in the sample.
- Print both MSE values and the r / ϵ values used, and save the 1x2 comparison as `energy_vs_threshold_pruning.png`.

Practice Set L — Convergence Rate: Reconstruction Error vs. Harmonic Count N

Total Marks: 10 | Time: 25 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

This set studies how quickly the Fourier series converges to the true signal as the truncation order N grows, and asks you to fit and interpret the empirical convergence rate rather than just reading off a table.

Task 1: MSE-vs-N Sweep

For N in $\{5, 10, 20, 40, 80, 150, 250\}$, build a fresh `FourierEpicycles(t, z, n_harmonics=N)` each time (no pruning), compute all coefficients, and record `evaluate_reconstruction_error()`. Plot MSE versus N on log-log axes (`plt.loglog`) with markers.

Task 2: Empirical Convergence-Rate Fit

Fit a line to $(\log N, \log \text{MSE})$ using `np.polyfit(np.log(Ns), np.log(mses), 1)` to estimate the power-law exponent p such that $\text{MSE} \sim N^p$. Print the fitted p and annotate it on the log-log plot (e.g. via `plt.text` or a legend label).

Execution & Deliverables

- Load `svgs/heart.svg` (`num_points=1000`) exactly as the sample entry point does.
- Print a small table of N and MSE for every tested N plus the final fitted exponent p .
- Save the log-log convergence plot as `mse_vs_N_convergence.png`.

Practice Set M — Robustness of the Reconstruction to Coefficient Phase Noise

Total Marks: 10 | Time: 25 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

This set tests understanding of amplitude vs. phase again, but through a noise-robustness lens: how much does the drawing degrade if every coefficient's PHASE (not magnitude) is perturbed by random noise of increasing strength?

Task 1: `perturb_phase(self, phase_std)`

Add a method that returns a NEW coefficients dict where every `c_n` is replaced by $|c_n| \cdot \exp(1j \cdot (\text{angle}(c_n) + \text{noise}_n))$, with `noise_n` drawn i.i.d. from `np.random.normal(0, phase_std, size=1)` for each harmonic `n` (use a fixed random seed for reproducibility), WITHOUT modifying `self.coefs`.

Task 2: Degradation Curve

For `phase_std` in `{0, 0.05, 0.1, 0.2, 0.4, 0.8}` radians, build the perturbed coefficients, reconstruct over `self.t` via `approximate(self.t, coeffs=perturbed)`, compute the MSE against the true signal, and plot MSE vs. `phase_std` (`plt.plot, marker='o'`). Also show the reconstructed curve at `phase_std = 0.2` and `phase_std = 0.8` side by side with the original in a 1x3 subplot to make the visual degradation concrete.

Execution & Deliverables

- Load `svgs/heart.svg` with `N = 150` as in the sample; leave `self.coefs` on the `FourierEpicycles` instance untouched throughout (all perturbation happens on local dict copies).
- Save the degradation curve as `phase_noise_degradation.png` and the 1x3 visual comparison as `phase_noise_visual_comparison.png`.

Practice Set N — Even/Odd Decomposition of the Traced Signal

Total Marks: 10 | Time: 25 Minutes | Files: `fs_redrawer.py` (template)

Context & Objective

Any periodic complex signal can be split into an even part and an odd part about $t=0$, and this decomposition has a clean Fourier-coefficient interpretation. This set asks you to implement the decomposition both in the time domain and via coefficient manipulation, and to verify they agree.

Task 1: Time-Domain Even/Odd Split

Add a method **`even_odd_split(self)`** that returns `(signal_even, signal_odd)` sampled on `self.t`, where $\text{signal_even}(t) = (f(t) + f(-t \bmod T)) / 2$ and $\text{signal_odd}(t) = (f(t) - f(-t \bmod T)) / 2$. Since `self.t` is uniform on a closed period, $f(-t \bmod T)$ can be obtained by indexing `self.signal` with `(len(t) - 1 - i)` for the i -th sample (with appropriate wraparound at the endpoints) rather than re-interpolating.

Task 2: Coefficient-Domain Verification

Build the coefficients of the even part directly from `self.coefs` as $c_{n_even} = (c_n + c_{-n}) / 2$ (and similarly $c_{n_odd} = (c_n - c_{-n}) / 2$), reconstruct both using `approximate(self.t, coefs=...)`, and compare each against the corresponding time-domain split from Task 1 by reporting the MSE between the two computations of the even part and between the two computations of the odd part (both MSEs should be extremely small, confirming the two derivations agree).

Task 3: Visualization

Produce a 1x3 subplot: original curve, even-part curve, odd-part curve (all as (Re, Im) plots), each using the coefficient-domain reconstruction from Task 2.

Execution & Deliverables

- Load `svgs/heart.svg` with $N = 150$ as in the sample.
- Print both verification MSEs (even-part agreement, odd-part agreement).
- Save the 3-panel figure as `even_odd_decomposition.png`.